カクラスに対するNP完全性とカードベースゼロ知識証明

池田 昇太[1,a)]　品川 和雅[2,3,b)]

概要：カクラスは、盤面上のいくつかのマスを塗り、塗られたマスに対応する番号の重みの総和が各行・各列に与えられた目標和と一致するようにするパズルである。本稿では、カクラスの解の存在判定問題がNP 完全であることを、部分和問題からの帰着によって示す。さらに、カクラスに対するカードベースゼロ知識証明プロトコルを提案する。提案プロトコルは、必要なシャッフル回数が盤面サイズに依存せず、2回のパイルスクランブルシャッフルのみで実行できる。

キーワード：カードベース暗号、ゼロ知識証明、NP 完全性、カクラス

NP-Completeness and Card-Based Zero-Knowledge Proofs for Kakurasu

Shota Ikeda[1,a)]　Kazumasa Shinagawa[2,3,b)]

Abstract: Kakurasu is a logic puzzle in which some cells of a grid are colored so that the sum of the weights assigned to the colored cells in each row and column matches the corresponding target value. In this paper, we prove that the decision problem of determining whether a Kakurasu instance has a solution is NP-complete by a reduction from the Subset Sum problem. We also propose a card-based zero-knowledge proof protocol for Kakurasu. Our protocol requires only two pile-scramble shuffles, independent of the size of the puzzle.

Keywords: Card-based cryptography, Zero-Knowledge Proofs, NP-Complete, Kakurasu

1. はじめに

1.1 カクラス

カクラス（Kakurasu）は、$n \times n$ の盤面上でマスを塗る配置系ロジックパズルである [1, 4]。盤面の各行および各列には目標和が与えられており、以下の条件を満たすようにマスを塗る。

(1) 各行では、塗られたマスの列番号（左から $1, 2, \ldots, n$）の和が、その行に与えられた目標和と一致する。
(2) 各列では、塗られたマスの行番号（上から $1, 2, \ldots, n$）の和が、その列に与えられた目標和と一致する。

すなわち、各マスは塗るか塗らないかを選択し、塗られたマスの位置に対応する番号を重みとして合計した値が、各行・各列の目標和と一致するように配置することが目的である。図 1 にカクラスの例題とその解答を示す。この図において、盤面の右に青字で書かれているのが各行の目標和であり、下に青字で書かれているのが各列の目標和である。

このパズルは、1989 年 1 月発行の『パズル通信ニコリ』第 24 号において「ボックス」の名称で初めて紹介されたとされる。当時は「四角ヌリ」とも呼ばれ、英語では「Index Sums」の名称でも知られている [13]。

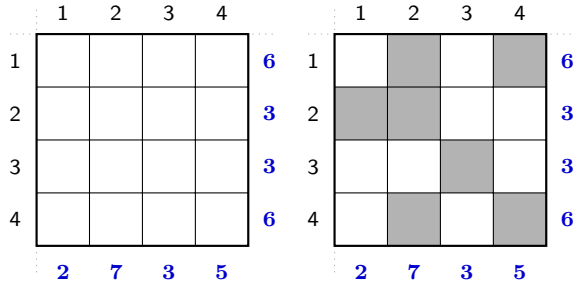
1.2 カードベースゼロ知識証明

ゼロ知識証明（Zero-Knowledge Proof: ZKP）[5] は、1985 年に提案された対話型証明方式であり、現代の暗号理論を支える重要な基盤技術の一つである。この方式では、ある問題に解が存在することを主張する証明者（Prover: P）と、その主張の正当性を確認する検証者（Verifier: V）の 2 者がプロトコルを実行する。証明者 P の目的は、解が存在

[1] 茨城大学 Ibaraki University
[2] 筑波大学 University of Tsukuba
[3] 産業技術総合研究所 National Institute of Advanced Industrial Science and Technology
[a)] 25nm709x@vc.ibaraki.ac.jp
[b)] shinagawa@cs.tsukuba.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.



(a) 例題　　(b) 解答

図 1: カクラスの例題・解答 (4×4)

することを検証者 V に納得させる一方で、その具体的な解に関する情報は一切開示しないことである。すなわち、証明対象となる問題に解が存在するという事実のみを伝え、それ以外の情報を検証者に与えないことがゼロ知識証明の本質である。

一般に、ゼロ知識証明は次の 3 つの性質を満たす証明方式として定義される。

(1) **完全性（Completeness）：** 証明者 P が問題に対する正当な解を持っており、プロトコルを誠実に実行した場合、検証者 V はその証明を必ず受理する。

(2) **健全性（Soundness）：** 証明者 P が問題に対する正当な解を持たない場合には、どのような不正な戦略を用いたとしても、検証者 V は証明を受理することはない。

(3) **ゼロ知識性（Zero-Knowledge）：** プロトコルの実行を通じて、検証者 V が得られる情報は「問題に解が存在する」という事実のみに限られ、その解に関する情報は一切得られない。

ゼロ知識証明は当初、計算機上で実行される暗号プロトコルとして研究が進められてきた。しかし、近年ではカードや天秤などの物理的な道具を用いて同様の性質を実現するプロトコルが数多く提案されている。その中でも、カードを用いて構成されるゼロ知識証明は、カードベースゼロ知識証明（Card-based Zero-Knowledge Proof）と呼ばれる。カードベースゼロ知識証明では、カードをめくる操作や並び替える操作、シャッフル操作を利用して秘密情報を秘匿したまま証明を行うため、複雑な計算や高度な数学的知識を必要としない。そのため、暗号プロトコルの教育や可視化、さらには物理的な環境におけるゼロ知識証明の実現方法として注目を集めている。その中で、プロトコル全体を通して必要なシャッフル回数が、パズルの問題サイズに依存しない定数となるプロトコルが複数提案されている。これまでに、数独 [17]、グラフ同型問題 [10]、ハミルトン閉路問題 [7] など、様々な問題を対象としたプロトコルが実現されている。

1.3 貢献

本稿では、ペンシルパズルであるカクラスの NP 完全性を、既知の NP 完全問題である部分和問題からの帰着によって証明する。また、カクラスに対するカードベースゼロ知識証明プロトコルを提案する。提案プロトコルは盤面サイズに依存せず、2 回のパイルスクランブルシャッフルによって実行可能である。

さらに、提案プロトコルを応用することで、Sumplete に対しても、わずか 2 回のパイルスクランブルシャッフルで実行可能なカードベースゼロ知識証明プロトコルを構成できる。既存のプロトコルでは、カード枚数は盤面の整数の大きさに依存し、$O(n^2 + \log k)$ 枚、シャッフル回数は $O(n^2 \log k)$ 回を要する。ここで、k は各行および各列における整数の絶対値の総和の最大値を表す。一方、本稿のプロトコルでは、カード枚数は $O(n^3)$ 枚であるものの、シャッフル回数を 2 回、すなわち $O(1)$ まで削減した。

2. NP 完全性の証明

本章では、カクラスの NP 完全性を証明する。まず、2.1 節で証明で用いるカクラスと部分和問題の定義を行う。そして、2.2 節でカクラスの NP 完全性を証明する。

2.1 問題の定義

本節では、証明で扱うカクラスと部分和問題を定義する。

定義 1 (カクラス)**.** 盤面サイズを $n \times n$ の正方行列とし、各マス (i, j) の状態を二値変数 $x_{i,j} \in \{0, 1\}$（1: 黒、0: 白）で表す。行 i の目標和を R_i、列 j の目標和を C_j としたとき、以下の制約を満たす割り当て $X = (x_{i,j})$ が存在するかを問う決定問題である。

$$\sum_{j=1}^{n} j \cdot x_{i,j} = R_i \quad (\forall i \in \{1, \ldots, n\}) \tag{1}$$

$$\sum_{i=1}^{n} i \cdot x_{i,j} = C_j \quad (\forall j \in \{1, \ldots, n\}) \tag{2}$$

定義 2 (部分和問題（Subset Sum Problem）)**.** 正の整数の有限集合 $S = \{a_1, a_2, \ldots, a_k\}$ と目標値 K が与えられたとき、要素の和が K となるような部分集合 $S' \subseteq S$（すなわち $\sum_{a \in S'} a = K$）が存在するかを問う決定問題である。

2.2 証明

定理 3. *カクラスはクラス NP に属する。*

証明. カクラスのインスタンスに対する証拠として、$n \times n$ の二値行列 $X = (x_{i,j}) \in \{0, 1\}^{n \times n}$ が与えられたとする。検証アルゴリズムは、与えられた X に対して各行の和 $\sum_{j=1}^{n} j \cdot x_{i,j}$ および各列の和 $\sum_{i=1}^{n} i \cdot x_{i,j}$ を計算し、それぞれが与えられた目標和 R_i, C_j と一致するかを確認する。

1 つの行または列の和を計算するための加算・乗算の回

© 2026 Information Processing Society of Japan

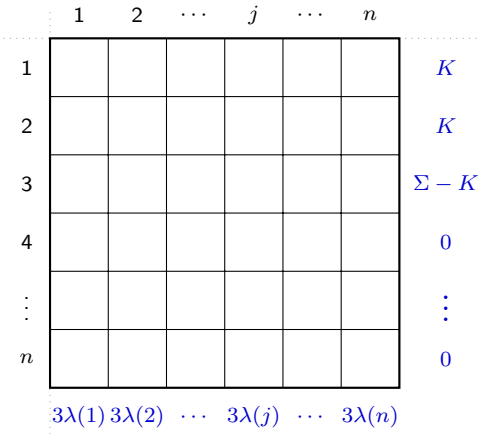


図 2: 部分和問題から構成されるカクラスの盤面。盤面の外側の数値は行目標和・列目標和を表す。

数は $O(n)$ 回である。行と列はそれぞれ n 個存在するため、すべての制約（式 1, 2）の検証にかかる総演算回数は $O(n^2)$ 回となる。これは入力サイズに対して多項式時間であるため、題意は示された。 □

定理 4. カクラスは *NP* 困難である。

証明. 既知の NP 完全問題である部分和問題から、カクラスへの多項式時間帰着を構成することで証明する。部分和問題の任意のインスタンス (S, K) が与えられたとする。ここで、S は正の整数の集合、K は目標値である。$\Sigma = \sum_{a \in S} a$ とおく。$M = \max_{a \in S} a$ とし、盤面サイズを $n = \max(3, M)$ とする（すなわち、$S \subseteq \{1, 2, \ldots, n\}$ とみなせる）。

ここで、各要素 $j \in \{1, \ldots, n\}$ に対し、関数 $\lambda(j)$ を「$j \in S$ ならば $\lambda(j) = 1$、そうでないならば $\lambda(j) = 0$」と定義する。このとき、図 2 のようにカクラスの $n \times n$ 盤面を用意し、各行の目標和 R_i と各列の目標和 C_j を以下のように設定する。

- 行目標和: $R_1 = K$, $R_2 = K$, $R_3 = \Sigma - K$, そして $i \geq 4$ に対しては $R_i = 0$ とする。
- 列目標和: 各列 $j \in \{1, \ldots, n\}$ に対して $C_j = 3\lambda(j)$ とする。

この構成にかかる時間は入力サイズに対して明らかに多項式時間である。

次に、この帰着の正当性（部分和問題が解を持つことと、構成したカクラスが解を持つことが同値であること）を示す。構成されたカクラスにおいて、i 行 j 列のマスが塗られる場合を $x_{i,j} = 1$、そうでない場合を $x_{i,j} = 0$ とする。正当性の証明の準備として、まず以下の補題を用意する。

補題 5. 構成されたカクラスのインスタンスが解 $X = (x_{i,j})$ を持つならば、任意の $j \in S$ において $(x_{1,j}, x_{2,j}, x_{3,j}) \in \{(1,1,0), (0,0,1)\}$ が成り立つ。

補題 *5* の証明. $i \geq 4$ の行の目標和は $R_i = 0$ である。カクラスのルール上、マスの状態は $x_{i,j} \in \{0, 1\}$ であり、列番号の重みは正であるため、解においては $x_{i,j} = 0$ $(i \geq 4)$ が確定する。したがって、$j \in S$ （すなわち $\lambda(j) = 1$）の列においては、列目標和 $C_j = 3\lambda(j) = 3$ の制約により、以下の方程式が成り立つ。

$$1 \cdot x_{1,j} + 2 \cdot x_{2,j} + 3 \cdot x_{3,j} = 3 \quad (x_{i,j} \in \{0, 1\}) \tag{3}$$

$x_{3,j} = 1$ の場合、残りの和は 0 となり $x_{1,j} = x_{2,j} = 0$ が確定する。$x_{3,j} = 0$ の場合、残りの和は 3 となり、$x_{1,j} = 1, x_{2,j} = 1$ が確定する。これ以外の解は存在しないため、補題は示された。 □

補題 5 を踏まえ、同値性を双方向から証明する。

(i) 十分性（部分和 YES ⇒ カクラス YES）:
部分和問題に解 $S' \subseteq S$ （$\sum_{j \in S'} j = K$）が存在すると仮定する。このとき、$X = (x_{i,j})$ を以下のように構成する。

- $j \in S'$ の列: $(x_{1,j}, x_{2,j}, x_{3,j}) = (1, 1, 0)$
- $j \in S \setminus S'$ の列: $(x_{1,j}, x_{2,j}, x_{3,j}) = (0, 0, 1)$
- それ以外のマス（$j \notin S$ または $i \geq 4$）: すべて 0

この割り当てにおいて、列制約 C_j は明らかに満たされる。行制約について確認すると、

$$\text{第 1 行}: \quad \sum_{j=1}^{n} j \cdot x_{1,j} = \sum_{j \in S'} j \cdot 1 = K = R_1$$

$$\text{第 2 行}: \quad \sum_{j=1}^{n} j \cdot x_{2,j} = \sum_{j \in S'} j \cdot 1 = K = R_2$$

$$\text{第 3 行}: \quad \sum_{j=1}^{n} j \cdot x_{3,j} = \sum_{j \in S \setminus S'} j \cdot 1 = \left(\sum_{j \in S} j \right) - K = R_3$$

となり、すべての行制約も満たされる。したがって構成したカクラスにも解が存在する。

(ii) 必要性（カクラス YES ⇒ 部分和 YES）:
構成したカクラスに解 $X = (x_{i,j})$ が存在すると仮定する。部分集合 S' を次のように構成する。

$$S' = \{ j \in S \mid x_{1,j} = 1 \} \tag{4}$$

第 1 行の目標和 $R_1 = K$ の制約より、

$$K = \sum_{j=1}^{n} j \cdot x_{1,j} = \sum_{j \in S} j \cdot x_{1,j} = \sum_{j \in S'} j \tag{5}$$

が成り立つ。これは S' が部分和問題の解であることを示している。したがって部分和問題にも解が存在する。

以上 (i), (ii) により帰着の正当性が示された。ゆえにカクラスは NP 困難である。 □

定理 6. カクラスは *NP* 完全である。

証明. 定理 3 によりカクラスはクラス NP に属し、定理 4 により NP 困難である。したがって、定義よりカクラスは NP 完全である。 □

© 2026 Information Processing Society of Japan

3. プロトコル構成に向けた準備

本章では提案プロトコルで使用するカードとシャッフル、サブプロトコルについて解説する。

3.1 使用するカード

本稿で提案するプロトコルでは、2 色カード（黒赤カード）を用いる。2 色カードは、カードベース暗号 [2,3] において一般的に用いられるカードであり、表面にはクラブ $\boxed{\clubsuit}$ またはハート $\boxed{\heartsuit}$ が描かれ、裏面はすべて $\boxed{?}$ である。また、本稿では 1 ビットの値を 2 枚のカードで表現する 2 枚符号化を採用する。2 枚符号化では，$x \in \{0,1\}$ に対し、

$$\boxed{\clubsuit}\boxed{\heartsuit} = 0, \qquad \boxed{\heartsuit}\boxed{\clubsuit} = 1$$

と符号化する。このように符号化された 2 枚組を**コミットメント**と呼ぶ。

さらに，数字カードを用いる。数字カードの表面には $\boxed{1}\boxed{2}\boxed{3}\cdots$ のような数字が描かれている。また、上線数字カードとは、数字の上に線が引かれた $\boxed{\overline{1}}\boxed{\overline{2}}\boxed{\overline{3}}\cdots$ のカードである。加えて，表面に何も描かれていないダミーカード $\boxed{}$ も用いる。これらのカードについても，2 色カードと同様に裏面はすべて $\boxed{?}$ である。

3.2 シャッフル

パイルスクランブルシャッフル（Pile-Scramble Shuffle）[8] は，カードの束 (pile) に対して一様ランダムな置換を適用することで実現されるシャッフルである。カード束の列 $(p_1, p_2, \ldots, p_n)$ があったとき、パイルスクランブルシャッフルは n 次対称群 S_n から置換 $\pi \in S_n$ を一様ランダムに選択し、その後 $(p_{\pi^{-1}(1)}, p_{\pi^{-1}(2)}, \ldots, p_{\pi^{-1}(n)})$ を出力する。カード束に対するパイルスクランブルシャッフルの適用は $[p_1|p_2|\cdots|p_n]$ で表記する。

3.3 Mizuki–Sone のコピープロトコル

本稿では、Mizuki–Sone のコピープロトコル [11] を用いる。このプロトコルは、入力 $x \in \{0,1\}$ と k 組の値 0 のコミットメントから、値を秘匿したまま x のコミットメントを k 個複製するプロトコルである。また，本プロトコルではランダム二等分割カット（Random Bisection Cut）を 1 回用いる[*1]。

$$\underbrace{\boxed{?}\boxed{?}}_{x}\boxed{\clubsuit}\boxed{\heartsuit}\cdots\boxed{\clubsuit}\boxed{\heartsuit} \to \boxed{\clubsuit}\boxed{\heartsuit}\underbrace{\boxed{?}\boxed{?}}_{x}\cdots\underbrace{\boxed{?}\boxed{?}}_{x}$$

なお、このコピープロトコルでは、各コミットメントに識別子となる数字カードを付与することで、複数の入力 $x_1, x_2, \ldots, x_n \in \{0,1\}$ のコミットメントを、1 回のパイルスクランブルシャッフルにより同時に複製できる。このように、通常は個別に行う複数回のシャッフルを、識別子となるカードを用いることで 1 回のシャッフルに集約して実行する手法は、バッチング手法 [15] と呼ばれる。そして、各入力コミットメントは、それぞれ任意個複製することが可能である。例として、2 つのコミットメント $a, b \in \{0,1\}$ を、それぞれ 2 個および 3 個にコピーする手順を示す。

(1) コミットメント a, b がそれぞれ与えられている。

$$\underbrace{\boxed{?}\boxed{?}}_{a}\underbrace{\boxed{?}\boxed{?}}_{b}$$

(2) 各コミットメントの下に、必要なコピー数に応じて 0 のコミットメントを配置する。この例では、a については 2 個、b については 3 個となるように配置する。

$$\begin{array}{cccc}\boxed{?} & \boxed{?} & \boxed{?} & \boxed{?}\\ \boxed{\clubsuit} & \boxed{\heartsuit} & \boxed{\clubsuit} & \boxed{\heartsuit}\\ \boxed{\clubsuit} & \boxed{\heartsuit} & \boxed{\clubsuit} & \boxed{\heartsuit}\\ & & \boxed{\clubsuit} & \boxed{\heartsuit}\end{array}$$

(3) 各コミットメントに識別用の数字カードを付与する。具体的には、a のコミットメントには $\boxed{1}$、b のコミットメントには $\boxed{2}$ を付ける。また、各カード束の枚数を揃えるため、a のカード束にはダミーカード $\boxed{}$ を追加する。

$$\begin{array}{cccc}\boxed{1} & \boxed{1} & \boxed{2} & \boxed{2}\\ \boxed{?} & \boxed{?} & \boxed{?} & \boxed{?}\\ \boxed{\clubsuit} & \boxed{\heartsuit} & \boxed{\clubsuit} & \boxed{\heartsuit}\\ \boxed{\clubsuit} & \boxed{\heartsuit} & \boxed{\clubsuit} & \boxed{\heartsuit}\\ \boxed{} & \boxed{} & \boxed{\clubsuit} & \boxed{\heartsuit}\end{array}$$

(4) すべてのカードを裏向きにした後、各列を 1 つのカード束とみなし、パイルスクランブルシャッフルを適用する。

$$\left[\begin{array}{c|c|c|c}\boxed{?} & \boxed{?} & \boxed{?} & \boxed{?}\\ \boxed{?} & \boxed{?} & \boxed{?} & \boxed{?}\\ \boxed{?} & \boxed{?} & \boxed{?} & \boxed{?}\\ \boxed{?} & \boxed{?} & \boxed{?} & \boxed{?}\\ \boxed{?} & \boxed{?} & \boxed{?} & \boxed{?}\end{array}\right]$$

(5) 上から 2 段のカードをすべて表にし、カード束を識別子の順に並べ替える。

$$\begin{array}{cccc}\boxed{1} & \boxed{1} & \boxed{2} & \boxed{2}\\ \boxed{\clubsuit} & \boxed{\heartsuit} & \boxed{\clubsuit} & \boxed{\heartsuit}\\ \boxed{?} & \boxed{?} & \boxed{?} & \boxed{?}\\ \boxed{?} & \boxed{?} & \boxed{?} & \boxed{?}\\ \boxed{?} & \boxed{?} & \boxed{?} & \boxed{?}\end{array}$$

このとき、$\boxed{1}$ が付いたカード束では、3 段目と 4 段目のカード対が a のコミットメントとなる。同様に、$\boxed{2}$ が付いたカード束では、3 段目から 5 段目までのカー

[*1] ランダム二等分割カットは，パイルスクランブルシャッフル（3.2 節）においてカード束が 2 個の場合に相当する。

© 2026 Information Processing Society of Japan

ド対が b のコミットメントとなる。

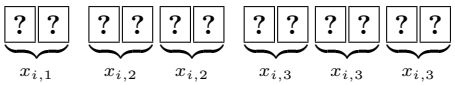
以上の手順を一般化することで、任意個のコミットメントを、それぞれ所望の個数だけ、1 回のパイルスクランブルシャッフルによって同時にコピーできる。なおダミーカードは、カード束ごとの枚数が異なることを許す一般化パイルスクランブルシャッフル [9] を用いる場合には不要である。

4. 提案プロトコル

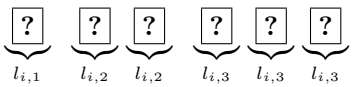
本章では、カクラスに対するカードベースゼロ知識証明プロトコルを提案する。まず、4.1 節で提案プロトコルのアイディアを示し、4.2 節で提案プロトコルの手順を解説する。そして、4.3 節で提案プロトコルの正当性を証明し、最後の 4.4 節で提案プロトコルの性能を評価する。

4.1 プロトコルのアイディア

本節では、提案プロトコルのアイディアを説明するため、3×3 のカクラス盤面における第 i 行（$1 \le i \le 3$）を考える。このとき、第 i 行の各マスに対応するコミットメント $x_{i,1}, x_{i,2}, x_{i,3}$ が、それぞれ 1 個、2 個、3 個存在するとする。



次に、各コミットメント $x_{i,j}$ から左側のカード $l_{i,j}$ のみを取り出す。すると、次のカード列が得られる。



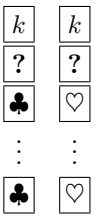
このカード列に対して完全シャッフル（各カード束が 1 枚のカードで構成されるパイルスクランブルシャッフル）を適用した後、すべてのカードを表向きにする。このとき、♡ のカードがちょうど R_i 枚（R_i は第 i 行における目標和）であれば、第 i 行はカクラスの条件を満たしていることが確認できる。この検証方法は一般の盤面にも自然に拡張できる。すなわち、すべての行および列に対して同様の検証を行うことで、盤面全体がカクラスの条件を満たしていることを検証できる。

4.2 プロトコルの手順

本節では、提案プロトコルの手順を説明する。証明者 P は、自身が知るカクラスの解 $X = (x_{i,j})$ に基づいて、以下の手順を実行する。

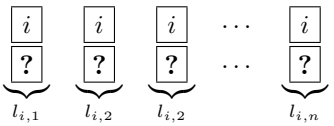
(1) **入力：**証明者 P は、解 $X = (x_{i,j})$ に従って、各マスに対応するコミットメントを裏向きに配置する。具体的には、$x_{i,j} = 1$ なら ♡♣、$x_{i,j} = 0$ なら ♣♡ を配置し，その内容は検証者 V には秘匿される。

(2) **コピー：**各マス (i, j) のコミットメントに対して、バッチング技術を適用した Mizuki–Sone のコピープロトコル（3.3 節）を実行し、$\max(i, j)$ 個のコピーを生成する。コピーには、コミットメントを識別するための数字カード k $(k = (i-1)n + j)$ を 2 枚と、0 のコミットメントを $\max(i, j)$ 個用いる。

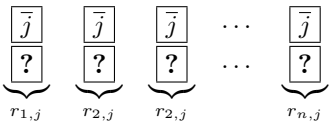


なお、$x_{1,1}$ は $\max(i, j) = 1$ であるため、コピーを行う必要がないことに注意されたい。

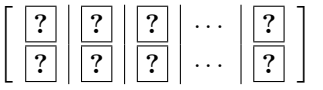
(3) **行制約および列制約の検証：**まず、各行 $i \in \{1, \ldots, n\}$ について、第 i 行のコミットメント $x_{i,1}, x_{i,2}, \ldots, x_{i,n}$ の左側のカード $l_{i,j}$ $(1 \le j \le n)$ を各 j に対して j 枚ずつ取り出す。取り出した各カードには、その行を識別するための数字カード i を付ける。



同様に、各列 $j \in \{1, \ldots, n\}$ について、第 j 列のコミットメント $x_{1,j}, x_{2,j}, \ldots, x_{n,j}$ の右側のカード $r_{i,j}$ $(1 \le i \le n)$ を各 i に対して i 枚ずつ取り出す。列については、行と区別するため、識別子として上線数字カード $\overline{j}$ を用いる[*2]。



次に、すべての識別子カードを裏向きにする。識別子カードとその直下のカードを 1 つの束とみなし、それら全体に対してパイルスクランブルシャッフルを適用する。



シャッフル後、すべてのカードを表向きにする。各行 i については ♡ の枚数がちょうど R_i 枚であることを確認し、各列 j については ♣ の枚数がちょうど C_j 枚であることを確認する。すべての行および列でこれらの条件が満たされれば、検証者 V は証明を受理し、そうでなければ棄却する。

4.3 プロトコルの正当性

本節では、提案プロトコルがゼロ知識証明の正当性を満

[*2] 列の識別子として $j' = j + n$ に対応する数字カードを用いてもよいが、説明を簡潔にするため、本稿では上線数字カードを用いる。

© 2026 Information Processing Society of Japan

たすための三つの条件である完全性、健全性、およびゼロ知識性をすべて満たすことを示す。

完全性　証明者 P がカクラスの正しい解 $X = (x_{i,j})$ を知っており、その解に従ってプロトコルを実行するとする。手順（1）では、各マスに対応するコミットメントが解 X に従って正しく配置される。手順（2）では、各コミットメントを $\max(i,j)$ 個コピーするのみであり、各マスのコミットメント $x_{i,j}$ が表す値は変化しない。手順（3）では、各行について左側のカードを、各列について右側のカードを取り出し、それぞれパイルスクランブルシャッフルを適用する。このとき、各行に含まれる $\boxed{\heartsuit}$ の枚数および各列に含まれる $\boxed{\clubsuit}$ の枚数は、解 X に対応する値のままである。解 X はカクラスの制約を満たしているため、各行では $\boxed{\heartsuit}$ がちょうど R_i 枚、各列では $\boxed{\clubsuit}$ がちょうど C_j 枚となる。したがって、証明者 P が正しい解に従ってプロトコルを実行すると、すべての検証で証明が棄却されないため、プロトコルは完全性を満たす。

健全性　証明者 P がカクラスの解を持たず、不正なコミットメントを用いて検証者 V を欺こうとする場合を考える。このとき、手順（3）で用いるコミットメントが表す配置は、カクラスのすべての行制約および列制約を同時に満たすことができない。そのため、手順（3）で確認される各行の $\boxed{\heartsuit}$ の枚数または各列の $\boxed{\clubsuit}$ の枚数の少なくとも一方が、対応する制約 R_i または C_j と一致しなくなる。したがって、検証者 V は少なくとも一つの制約違反を検出するため、プロトコルは健全性を満たす。

ゼロ知識性　提案プロトコルにおいて、行制約検証および列制約検証で公開される情報が、解の具体的な配置情報を漏洩しないことを示す。手順（2）では、各コミットメントをコピーするが、パイルスクランブルシャッフルにより、各マスのコミットメントの値が何であるかは秘匿される。手順（3）では、各行について取り出した左側のカード、および各列について取り出した右側のカードに対して、それぞれパイルスクランブルシャッフルを適用する。そのため、各カードは一様ランダムに並び替えられた後に表にされる。したがって、各行では $\boxed{\heartsuit}$ がちょうど R_i 枚存在すること、各列では $\boxed{\clubsuit}$ がちょうど C_j 枚存在することは確認できるが、それらのカードが元の盤面のどのマスに対応するかという情報は、パイルスクランブルシャッフルによって完全に秘匿される。すなわち、検証者 V が得られる情報は、各行および各列が制約を満たしているという事実のみであり、各マスの値に関する情報は一切漏洩しない。したがって、プロトコルはゼロ知識性を満たす。

4.4　プロトコルの評価

4.4.1　カード枚数

提案プロトコル全体で使用するカード枚数について、各手順における内訳を以下に示す。

手順（1）　各マスに対応するコミットメントを配置するため、2 色カードを各マスにつき 2 枚、合計 $2n^2$ 枚用いる。

手順（2）　各マス (i,j) に対して、コピーに必要な値 0 のコミットメントを $\max(i,j)$ 個用意する。ただし、$(1,1)$ ではコピーを行わない。よって、コピーに必要な 2 色カードの総数は

$$2\left(\sum_{i=1}^{n}\sum_{j=1}^{n}\max(i,j)-1\right)=\frac{4n^3+3n^2-n-6}{3}$$

枚である。

また、各コミットメントを識別するための数字カードを各マスにつき 2 枚用いるため、数字カードは

$$2(n^2-1)$$

枚必要である。さらに、ダミーカードを用いる場合、必要な枚数は

$$\sum_{m=2}^{n}2(n-m)(2m-1)=\frac{(n-1)(n-2)(2n+3)}{3}$$

枚である。

手順（3）　各行について、識別子として数字カードを

$$\sum_{j=1}^{n}j=\frac{n(n+1)}{2}$$

枚用いるため、全 n 行で

$$\frac{n^2(n+1)}{2}$$

枚必要となる。同様に、上線数字カードも

$$\frac{n^2(n+1)}{2}$$

枚必要となる。

以上より、提案プロトコル全体で使用するカードの種類の内訳は以下の通りである。

2 色カード　$\dfrac{4n^3+9n^2-n-6}{3}$ 枚

数字カード　$\dfrac{n^3+5n^2-4}{2}$ 枚

上線数字カード　$\dfrac{n^2(n+1)}{2}$ 枚

ダミーカード　$\dfrac{(n-1)(n-2)(2n+3)}{3}$ 枚

よって、提案プロトコル全体で使用するカードの総数は

$$3n^3+5n^2-2n-2$$

枚である。

© 2026 Information Processing Society of Japan

4.4.2 シャッフル回数

提案プロトコル全体で使用するシャッフル回数の内訳は以下のとおりである。手順 (2) のコミットメントのコピーにおいて、$n+2$ 枚からなる計 $2n^2-2$ 個のカード束のそれぞれに対し、パイルスクランブルシャッフルを 1 回適用する。また、手順 (3) の検証において、2 枚からなる計 $n^2(n+1)$ 個のカード束のそれぞれに対し、パイルスクランブルシャッフルを 1 回適用する。よって、提案プロトコル全体でのシャッフル回数は計 2 回である。

5. Sumplete への応用

本章ではカクラスに対するカードベースゼロ知識証明プロトコルを用いて Sumplete というペンシルパズルに対してもカードベースゼロ知識証明を行えることを示す。

5.1 Sumplete

Sumplete [16] は、$n \times n$ の盤面上に配置された整数から、一部の数を取り除く配置系ロジックパズルである。盤面の各行および各列には目標和が与えられており、以下の条件を満たすように数を取り除く。

(1) 各行では、取り除かれずに残った数の総和が、その行に与えられた目標和と一致する。
(2) 各列では、取り除かれずに残った数の総和が、その列に与えられた目標和と一致する。

すなわち、各マスに配置された数について、取り除くか残すかを選択し、残された数の総和が各行・各列の目標和と一致するように配置を決定することが目的である。

Sumplete は、2023 年 3 月に OpenAI が開発した生成 AI である ChatGPT [12] によって考案された論理パズルである。Sumplete の解の存在判定問題は NP 完全であることが示されており [6]、盤面で使用される整数が 2 種類のみであっても、その判定問題は依然として NP 完全であることが知られている [14]。Sumplete の公式ルールでは、各マスに配置する整数に制約はなく、任意の整数を用いることができる。一方、公式 Web サイト [16] で公開されている問題では、すべての整数 m が $0<|m|<20$ を満たしている。本稿では、制限のない問題を Sumplete とし、ある整数 C に対して $0<|m|<C$ を満たす Sumplete を**制限付き Sumplete（restricted Sumplete）**とする。

5.2 Sumplete に対するカードベースゼロ知識証明

Hatsugai et al. [6] は、Sumplete に対するカードベースゼロ知識証明プロトコルを 2 種類提案した。整数符号化 [3] を用いたプロトコルでは、各行および各列に含まれる整数の絶対値の総和の最大値を k とすると、カード枚数は $O(n^2+k)$、シャッフル回数は $O(n^2)$ である。また、整数を 2 進表現するプロトコルでは、カード枚数は $O(n^2+\log k)$ まで削減できる一方、シャッフル回数は $O(n^2 \log k)$ となる。これに対して、本稿で提案する Sumplete に対するカードベースゼロ知識証明プロトコルでは、カード枚数は $O(n^3)$ であるものの、バッチング技法を用いることで、シャッフル回数を 2 回、すなわち $O(1)$ まで削減できる。

提案プロトコルの概要を以下に示す。まず、証明者 P は、取り除くマスには $\boxed{\clubsuit}\boxed{\heartsuit}$、取り除かないマスには $\boxed{\heartsuit}\boxed{\clubsuit}$ となるようにコミットメントを入力する。続いて、各マスのコミットメントを、そのマスに記載された整数の値だけ複製する。その後、カクラスの場合と同様に、各行については各コミットメントの左側のカードを、各列については右側のカードをそれぞれ取り出し、シャッフルして公開することで、行および列の目標和を満たしていることを検証できる。

このプロトコルを制限付き Sumplete に適用する場合、各マスの整数 m は $0<|m|<C$ を満たすため、必要なカード枚数はカクラスの場合と同様に $O(n^3)$ である。一方、通常の Sumplete では各マスの整数に上限がないため、整数の最大絶対値を M とすると、必要なカード枚数は $O(n^3+n^2M)$ となる。

5.3 負の整数を含む Sumplete の変換

盤面に負の整数が含まれる場合、すべてのマスに定数を加算する素朴な手法では、行および列ごとに選択されるマスの数が事前には分からないため、新しい目標和を一意に決定することができない。そこで、負の整数が配置されたマスについて「残す」と「消す」の対応を反転することで、問題の構造を維持したまま、すべての整数を正の整数に変換する。

具体的には、ある行（または列）を構成する各マスの整数を $g_1, g_2, \ldots, g_n$、目標和を S、各マスの選択状態を表す変数を $x_i \in \{0,1\}$（$x_i=1$ を「残す」、$x_i=0$ を「消す」）とする。このとき、満たすべき制約条件は次式で与えられる。

$$\sum_{i=1}^{n} g_i x_i = S \tag{6}$$

ここで、負の整数 $g_k<0$ が存在する場合、そのマスについて「残す」と「消す」の対応を反転させた新たな変数 $y_k \in \{0,1\}$ を導入し、

$$x_k = 1 - y_k$$

と定める。これを制約式に代入して整理すると、以下の等価な条件式が得られる。

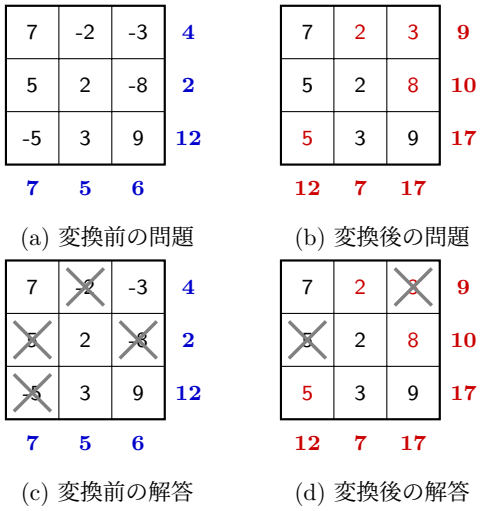
$$\sum_{i \neq k} g_i x_i + (-g_k) y_k = S - g_k \tag{7}$$

$g_k<0$ であるため、変換後の整数 $-g_k$ は正の整数となる。また、右辺の新しい目標和 $S-g_k$ は定数として一意に計算可能である。この操作を盤面内のすべての負の整数に

© 2026 Information Processing Society of Japan

対して適用することにより、盤面上のすべての整数を正の整数とする、元の問題と論理的に等価な問題インスタンスを生成できる。この変換による問題インスタンスおよび解の変化の例を図 3 に示す。

本プロトコルは、この変換後の盤面に対して実行される。実行後、元の盤面において負の整数が配置されていたマスについては、得られた変数 y_k に応じて「残す」と「消す」の解釈を反転させることで、元の問題における選択状態 x_k を復元できる。



(a) 変換前の問題　(b) 変換後の問題

(c) 変換前の解答　(d) 変換後の解答

図 3: 負の整数を含む Sumplete の変換過程と解答の変化。図 (b) および (d) の赤字は、絶対値化された整数および再計算された目標和を示す。図 (c) および (d) における × 印は削除マスを表しており、元の問題において負の整数が配置されていたマスでは、「残す」と「消す」の対応が反転している。

6. おわりに

本稿ではカクラスというペンシルパズルに対する NP 完全性の証明とカードベースゼロ知識証明プロトコルを提案した。特にカードベースゼロ知識証明プロトコルは、盤面サイズに依存しない定数回のシャッフルで構成できた。今後の研究課題としては、他のペンシルパズルに対して問題サイズに依存しない定数回シャッフルのプロトコルを構成すること、カクラスのような複数の整数の和を計算する必要があるペンシルパズルに対して、提案プロトコルの様な手法が応用できるかを検討することなどが挙げられる。

謝辞　本研究は JSPS 科研費 JP21K17702, JP26K21162 と JST CREST JPMJCR22M1 の支援を受けた。

参考文献

[1] Kakurasu. `https://ja.puzzle-kakurasu.com/`. Japanese version. Accessed: 2026-07-02.

[2] B. D. Boer. More efficient match-making and satisfiability the five card trick. In J.-J. Quisquater and J. Vandewalle eds., *EUROCRYPT 1989*, Vol. 434 of *LNCS*, pp. 208–217, Heidelberg, 1990. Springer.

[3] C. Crépeau and J. Kilian. Discreet solitary games. In D. R. Stinson ed., *Advances in Cryptology—CRYPTO' 93*, Vol. 773 of *LNCS*, pp. 319–330, Berlin, Heidelberg, 1994. Springer.

[4] Cross+A. パズル（カクラス）. `https://www.cross-plus-a.com/jp/puzzles.htm#Kakurasu`. Accessed: 2026-07-02.

[5] S. Goldwasser, S. Micali, and C. Rackoff. The knowledge complexity of interactive proof-systems (extended abstract). In *Proceedings of the 17th Annual ACM Symposium on Theory of Computing*, pp. 291–304, 1985.

[6] K. Hatsugai, S. Ruangwises, K. Asano, and Y. Abe. Np-completeness and physical zero-knowledge proofs for sumplete, a puzzle generated by chatgpt. *New Generation Computing*, 42(3):429–448, 2024.

[7] R. Igari, S. Odaka, Y. Komano, and T. Mizuki. Efficient physical ZKP protocols for Hamiltonian cycle problem and traveling salesman problem. *IEICE Trans. Fundam.*, 2026. In Press.

[8] R. Ishikawa, E. Chida, and T. Mizuki. Efficient card-based protocols for generating a hidden random permutation without fixed points. In C. S. Calude and M. J. Dinneen eds., *Unconventional Computation and Natural Computation*, Vol. 9252 of *LNCS*, pp. 215–226, Cham, 2015. Springer.

[9] Y. Ishizaki, K. Shinagawa, and K. Nuida. Batching techniques using generalized pile-scramble shuffles and branching shuffles. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, E109-A(9), Sept. 2026.

[10] D. Miyahara, L. Robert, P. Lafourcade, S. Takeshige, T. Mizuki, K. Shinagawa, A. Nagao, and H. Sone. Card-based ZKP protocols for Takuzu and Juosan. In M. Farach-Colton, G. Prencipe, and R. Uehara eds., *Fun with Algorithms*, Vol. 157 of *LIPIcs*, pp. 20:1–20:21, Dagstuhl, Germany, 2020. Schloss Dagstuhl.

[11] T. Mizuki and H. Sone. Six-card secure AND and four-card secure XOR. In X. Deng, J. E. Hopcroft, and J. Xue eds., *Frontiers in Algorithmics*, Vol. 5598 of *LNCS*, pp. 358–369, Berlin, Heidelberg, 2009. Springer.

[12] OpenAI. ChatGPT. `https://chatgpt.com/`, 2026. Accessed: 2026-07-17.

[13] Puzzle Pedia. ボックス. `https://scrapbox.io/puzzle-pedia/%E3%83%9C%E3%83%83%E3%82%AF%E3%82%B9`. Accessed: 2026-07-02.

[14] S. Ruangwises. Sumplete is hard, even with two different numbers. *arXiv*, 2023.

[15] K. Shinagawa and K. Nuida. A single shuffle is enough for secure card-based computation of any Boolean circuit. *Discrete Applied Mathematics*, 289:248–261, 2021.

[16] D. Tait. Sumplete. `https://sumplete.com/`, 2023. Accessed: 2026-07-17.

[17] K. Tanaka, S. Sasaki, K. Shinagawa, and T. Mizuki. Only two shuffles perform card-based zero-knowledge proof for sudoku of any size. In *2025 Symposium on Simplicity in Algorithms (SOSA)*, pp. 94–107. SIAM, 2025.

© 2026 Information Processing Society of Japan